Juego con obstáculos en ESP32 + OLED

Ismael Restrepo Prisco - Leidy Dahiana Espinal Ángel
Electrónica Digital II

Índice

1. Identificación del proyecto **2**

2. Resumen **2**

3. Descripción del hardware **2**

4. Descripción del software **3**

5. Estructura del código **4**

6. Explicación de funciones principales **5**

7. Comunicación **6**

8. Interfaz de usuario **7**

9. Procedimiento de prueba **7**

10. Manejo de errores y seguridad **8**

1. Identificación del proyecto

- **Nombre del proyecto:** Juego con obstáculos en ESP32 + OLED y MicroPython
- **Integrantes del equipo: Ismael Restrepo Prisco y Leidy Dahiana Espinal Ángel**

2. Resumen

En este proyecto se diseñó e implementó un videojuego interactivo tipo dodger utilizando un microcontrolador ESP32 programado en MicroPython, con salida gráfica en una pantalla OLED SSD1306 de 128×64 píxeles. El sistema permite al usuario controlar un personaje representado mediante un sprite, cuyo objetivo es evitar obstáculos generados dinámicamente que se desplazan a través de la pantalla.

El programa está estructurado mediante una máquina de estados que gestiona las diferentes etapas del juego: menú principal, ejecución del juego, pausa, fin de partida (game over) y condición de victoria en el modo contrarreloj. La interacción del usuario se realiza mediante tres pulsadores (arriba, abajo y start), los cuales permiten seleccionar modos de juego, controlar el movimiento del personaje y pausar o reanudar la partida.

Se implementan tres modos de juego: clásico, donde la dificultad aumenta progresivamente con el tiempo; contrarreloj, donde el objetivo es obtener la mayor cantidad de puntos en un tiempo limitado; y modo hardcore, caracterizado por una mayor velocidad y frecuencia de aparición de obstáculos. El sistema incluye generación de puntaje basada en la evasión de obstáculos, así como escalado dinámico de dificultad mediante variaciones en la velocidad y el tiempo de aparición de los mismos.

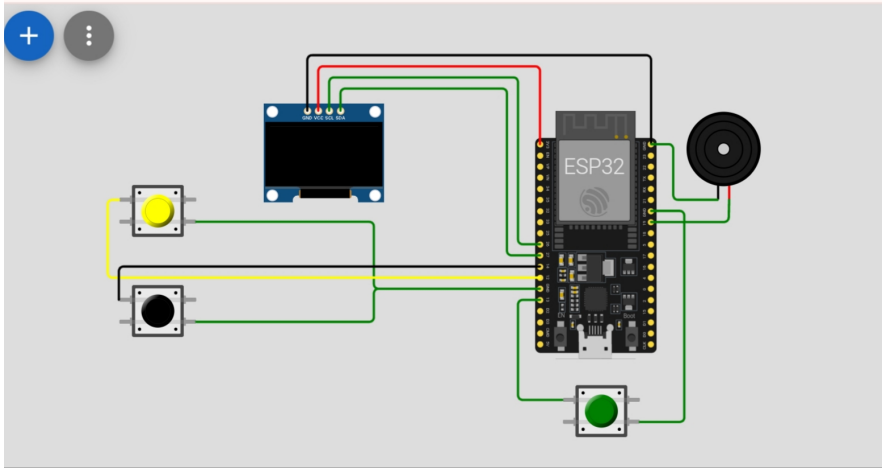
Adicionalmente, se incorporan efectos sonoros mediante modulación por ancho de pulso (PWM) para proporcionar retroalimentación al usuario en eventos como saltos, colisiones, selección de opciones y música en el modo contrarreloj. El manejo del tiempo se realiza utilizando funciones de temporización en milisegundos, lo que permite una ejecución fluida del juego cercana a 30 cuadros por segundo.

3. Descripción del hardware

- **ESP32**: Microcontrolador principal del sistema. Se utiliza debido a su capacidad de procesamiento, múltiples pines GPIO y soporte para periféricos como I2C y PWM. En este proyecto, el ESP32 ejecuta el programa en MicroPython, gestiona la lógica del juego, procesa las entradas de los botones, controla la pantalla OLED y genera las señales de audio para el buzzer.
- **Pantalla OLED SSD1306 (128×64)**: Se emplea como interfaz gráfica del sistema. Está conectada al ESP32 mediante comunicación I2C utilizando los pines GPIO 22 (SCL) y GPIO 21 (SDA). Esta pantalla permite mostrar texto, gráficos y animaciones del juego, incluyendo el menú, el personaje (sprite), los obstáculos y el puntaje en tiempo real.

- **Pulsadores**: Se utilizan tres botones conectados a los pines GPIO 12, 14 y 13, configurados como entradas digitales con resistencias internas pull-up. Su función es permitir la interacción del usuario con el sistema:
  - Botón UP (GPIO 12): Movimiento hacia arriba o navegación en el menú.
  - Botón DOWN (GPIO 14): Movimiento hacia abajo o navegación en el menú.
  - Botón START/PAUSE (GPIO 13): Iniciar el juego, pausar o reanudar.
- **Buzzer pasivo (piezoeléctrico)**: Conectado al pin GPIO 19 y controlado mediante PWM, se utiliza para generar efectos sonoros. A través de la variación de la frecuencia y el ciclo de trabajo, se producen diferentes tonos que representan eventos del juego como saltos, colisiones, victoria, pausa y música de fondo en el modo contrarreloj.
- **Protoboard y cables de conexión**: Se utilizan para realizar las conexiones físicas entre los componentes (ESP32, pantalla, botones y buzzer), permitiendo una implementación flexible y sin necesidad de soldadura.

Diagrama de conexión:



4. Descripción del software

El software del sistema fue desarrollado en MicroPython para el microcontrolador ESP32, y tiene como objetivo principal gestionar la lógica del videojuego, la interacción con el usuario y el control de los periféricos en tiempo real. El programa integra funcionalidades de lectura de entradas digitales, generación de señales PWM, comunicación I2C y renderizado gráfico en una pantalla OLED.

El código se ejecuta dentro de un bucle principal (loop) que opera aproximadamente a 30 cuadros por segundo, utilizando funciones de temporización en milisegundos para garantizar una ejecución fluida y controlada. Este enfoque permite sincronizar correctamente la actualización de la pantalla, la generación de obstáculos y la respuesta a las acciones del usuario.

El sistema está estructurado mediante una máquina de estados que define el comportamiento del juego en cada momento. Los estados implementados son: menú (selección de modo), juego activo, pausa, fin de partida (game over) y victoria en el modo contrarreloj. Esta organización permite un control claro del flujo del programa y facilita la transición entre diferentes etapas del juego.

El software gestiona la entrada del usuario mediante tres pulsadores, implementando detección de flanco para identificar cuándo un botón es presionado, evitando lecturas repetidas. A partir de estas entradas, el jugador puede desplazarse verticalmente, navegar en el menú y controlar la pausa del juego.

La lógica del juego incluye la generación dinámica de obstáculos, los cuales se crean de forma periódica con ayuda de funciones de tiempo y valores aleatorios. Estos obstáculos se desplazan horizontalmente y su frecuencia y velocidad varían dependiendo del modo de juego seleccionado. Además, se implementa un sistema de detección de colisiones basado en cajas delimitadoras (AABB), permitiendo determinar de manera eficiente cuándo el jugador impacta un obstáculo.

El movimiento del jugador se controla mediante dos enfoques: en los modos clásico y hardcore se implementa una simulación básica de gravedad y salto, mientras que en el modo contrarreloj el movimiento es vertical continuo y suavizado. Esta diferenciación permite ofrecer experiencias de juego variadas.

El sistema también incluye un módulo de audio basado en PWM para la generación de efectos sonoros y música simple, donde se controlan frecuencias y tiempos de activación del buzzer para representar distintos eventos del juego.

Finalmente, el software gestiona la visualización en la pantalla OLED, dibujando el personaje, los obstáculos, el menú y la información del juego (puntaje, tiempo, récord). Todo esto se realiza mediante funciones de dibujo que actualizan la pantalla en cada ciclo del programa.

5. Estructura del código

El código se encuentra organizado en diferentes secciones que facilitan su comprensión:

- Importación de librerías: Se importan los módulos necesarios para el funcionamiento del sistema, incluyendo control de pines (Pin), comunicación I2C, generación de señales PWM, temporización en milisegundos, manejo de la pantalla OLED (ssd1306) y generación de valores aleatorios (random).

- Configuración de periféricos: Se inicializan los componentes de hardware: Pantalla OLED mediante I2C, pulsadores como entradas digitales con resistencias pull-up, buzzer configurado con PWM para la generación de sonido. Además, se incluyen ajustes adicionales como la verificación de métodos disponibles en la pantalla (fillrect).

- Manejo de entradas (botones): Se define la función readbuttons(), encargada de leer el estado de los pulsadores y detectar eventos de presión mediante comparación con estados anteriores (detección de flanco). Esto evita múltiples lecturas por una sola pulsación.

- Módulo de audio: Incluye funciones como beep(), mariolose(), mariowin(), pausesound() y jumpsound(), que generan diferentes efectos sonoros mediante el buzzer. También se implementa musicupdate() para reproducir una secuencia de notas en el modo contrarreloj.
- Definición de sprites y gráficos: Se declaran representaciones del personaje mediante arreglos de texto (CRAB1, CRAB2) y la función drawsprite(), que permite dibujarlos píxel a píxel en la pantalla OLED.
- Definición de estados del sistema: Se establecen constantes para la máquina de estados (MENU, GAME, PAUSE, OVER, WIN) y variables globales que controlan el estado actual del juego, el modo seleccionado y los nombres de los modos.
- Variables del jugador y del juego: Posición y movimiento del jugador (posición, velocidad, gravedad, salto), obstáculos (lista, tiempos de aparición), puntaje, velocidad del juego y temporización, control de animación y detección de récord en modo contrarreloj.
- Funciones principales del juego: reset(): reinicia variables al comenzar una nueva partida. spawn(): genera obstáculos de forma dinámica. collide(): detecta colisiones entre el jugador y los obstáculos. update(): actualiza la lógica del juego (movimiento, puntaje, dificultad, colisiones). draw(): renderiza todos los elementos gráficos en la pantalla. drawmenu() y drawpause(): muestran las interfaces correspondientes.
- Bucle principal (loop): Se implementa un ciclo infinito (while True) que controla la ejecución del sistema. Dentro de este bucle: Se regula la velocidad de ejecución (30 FPS). Se leen las entradas del usuario. Se evalúa el estado actual del sistema. Se ejecutan las funciones correspondientes según el estado (menú, juego, pausa, etc.). Se gestionan las transiciones entre estados.

6. Explicación de funciones principales

El programa implementa diversas funciones que encapsulan la lógica del juego, el manejo de entradas, la generación de sonido y la interacción con la pantalla. A continuación, se describen las funciones más relevantes:

- `read_buttons()`: Esta función se encarga de leer el estado de los tres pulsadores (UP, DOWN y START). Implementa detección de flanco, comparando el estado actual con el anterior, lo que permite identificar el momento exacto en que un botón es presionado. Esto evita múltiples activaciones por una sola pulsación y mejora la precisión del control.
- `beep(f, d)`: Genera un sonido en el buzzer utilizando PWM. Recibe como parámetros la frecuencia (f) y la duración (d), permitiendo producir distintos tonos. A partir de esta función se construyen efectos más complejos.
- `mario_lose(), mario_win(), pause_sound(), jump_sound()`: Son funciones auxiliares que combinan múltiples llamadas a beep() para generar efectos sonoros específicos asociados a eventos del juego, como perder, ganar, pausar o saltar.

- `music_update(now)`: Permite reproducir una secuencia de notas musicales en el modo contrarreloj. Utiliza temporización basada en milisegundos para cambiar de nota periódicamente sin bloquear la ejecución del programa, logrando un efecto de música de fondo.
- `draw_sprite(s, x, y)`: Dibuja el personaje en la pantalla OLED a partir de una matriz de bits representada como texto. Recorre cada fila y columna del sprite, encendiendo los píxeles correspondientes.
- `reset()`: Reinicia todas las variables del juego al iniciar una nueva partida. Restablece la posición del jugador, la lista de obstáculos, el puntaje, la velocidad y los tiempos de referencia. Además, ajusta parámetros dependiendo del modo de juego seleccionado.
- `spawn(now)`: Genera nuevos obstáculos en función del tiempo transcurrido. Dependiendo del modo de juego, crea diferentes tipos de obstáculos (bloques en el suelo o tubos con espacio intermedio) y ajusta el tiempo entre apariciones de manera aleatoria, aumentando la variabilidad del juego.
- `collide(px, py, ox, oy, ow, oh)`: Implementa la detección de colisiones utilizando el método de cajas delimitadoras (AABB). Evalúa si el jugador y un obstáculo se superponen en pantalla. Incluye un pequeño ajuste (offset) para hacer la colisión más precisa visualmente.
- `draw()`: Renderiza todos los elementos visuales del juego en la pantalla OLED. Dibuja el jugador, los obstáculos, el suelo (si aplica), y la información en pantalla como puntaje y tiempo. También gestiona la animación del sprite alternando entre dos imágenes.
- `draw_menu()`: Muestra el menú principal del juego, permitiendo al usuario seleccionar el modo de juego mediante una interfaz simple con texto y un indicador de selección.
- `draw_pause()`: Muestra la pantalla de pausa, indicando al usuario cómo reanudar la partida.

7. Comunicación

El sistema implementado no utiliza protocolos de comunicación con dispositivos externos para intercambio de datos. Sin embargo, sí hace uso de comunicación interna entre el microcontrolador ESP32 y sus periféricos, principalmente mediante el protocolo I2C.

La comunicación más relevante es la establecida con la pantalla OLED SSD1306, la cual se realiza a través del bus I2C utilizando dos líneas: SCL (GPIO 22) y SDA (GPIO 21). A través de este protocolo, el ESP32 envía los datos necesarios para la representación gráfica del juego, incluyendo texto, sprites, obstáculos y elementos de la interfaz. Esta comunicación permite actualizar la pantalla de manera eficiente en cada ciclo del programa, manteniendo una visualización fluida del juego.

Por otro lado, la interacción con los pulsadores no se considera un sistema de comunicación como tal, sino una lectura directa de entradas digitales mediante los pines GPIO configurados con resistencias internas pull-up. De igual manera, la salida hacia el buzzer se realiza mediante

una señal PWM generada por el ESP32, la cual no implica un protocolo de comunicación, sino una modulación de señal para generar sonido.

8. Interfaz de usuario

La interfaz de usuario del sistema es de tipo gráfica y está implementada mediante una pantalla OLED SSD1306 de 128×64 píxeles, la cual permite visualizar en tiempo real todos los elementos del juego. Esta interfaz está diseñada para ser simple, clara e intuitiva, facilitando la interacción del usuario con el sistema a través de los pulsadores.

El sistema cuenta con un menú principal que se muestra al iniciar el programa, donde el usuario puede seleccionar entre los diferentes modos de juego (Clásico, Time y Hardcore). La navegación se realiza utilizando los botones UP y DOWN, mientras que la selección se confirma con el botón START. La opción seleccionada se indica mediante un cursor (flecha), lo que permite identificar fácilmente el modo activo.

Durante la ejecución del juego, la pantalla muestra el escenario principal, donde se visualizan:

- El personaje controlado por el usuario (sprite animado).
- Los obstáculos en movimiento.
- El puntaje en tiempo real. En el modo contrarreloj, además del puntaje, se muestran el tiempo restante y el récord anterior, brindando mayor información al jugador.

El sistema también incluye diferentes pantallas de estado, las cuales mejoran la experiencia de usuario:

- `Pantalla de pausa`: Se activa al presionar el botón START durante el juego, mostrando un mensaje que indica cómo reanudar la partida.
- `Pantalla de Game Over`: Aparece cuando el jugador colisiona con un obstáculo, mostrando el puntaje final.
- `Pantalla de victoria (modo Time)`: Se presenta cuando el jugador completa el tiempo establecido, indicando el puntaje alcanzado.

La interfaz se complementa con retroalimentación sonora mediante el buzzer, que emite diferentes sonidos para acciones como selección en el menú, salto, pausa, colisión o logro de objetivos. Esto refuerza la interacción y mejora la experiencia del usuario.

9. Procedimiento de prueba

Para verificar el correcto funcionamiento del sistema, se siguieron los siguientes pasos:

1. Se verificaron las conexiones físicas del ESP32, la pantalla OLED, los pulsadores y el buzzer.

2. Se comprobó que el sistema iniciara correctamente mostrando el menú principal.
3. Se validó el funcionamiento de los botones UP, DOWN y START para navegación y control del juego.
4. Se verificó el correcto renderizado del jugador, obstáculos y texto en la pantalla OLED.
5. Se probaron los diferentes modos de juego para validar la lógica y el incremento de dificultad.
6. Se comprobó la correcta detección de colisiones entre el jugador y los obstáculos.
7. Se evaluó el funcionamiento del buzzer en los distintos eventos del juego.
8. Se validó la fluidez del sistema asegurando una ejecución cercana a 30 FPS.
9. Se verificaron las transiciones correctas entre los estados del sistema (menú, juego, pausa, fin).

10. Manejo de errores y seguridad

El sistema implementa mecanismos básicos para garantizar un funcionamiento estable durante la ejecución del juego.

Se utiliza detección de flancos en los pulsadores para evitar múltiples lecturas erróneas por una sola pulsación. Se controla la temporización mediante ticks_ms() y ticks_diff() para evitar bloqueos y mantener una ejecución fluida. Se limitan variables como velocidad, posición y generación de obstáculos para prevenir comportamientos fuera de rango. Se valida la posición del jugador para que no salga de los límites de la pantalla. Se maneja la lista de obstáculos eliminando aquellos fuera de pantalla, evitando sobrecarga de memoria. Se controla la colisión mediante un ajuste de hitbox que reduce errores en la detección. Se evita saturar el buzzer apagándolo después de cada sonido. Se implementa una máquina de estados que previene ejecuciones indebidas de funciones según el contexto.